

Music Genre Classification using Deep Learning

COMP6252 Coursework 1 – University of Southampton

Akash Abbigeri
ECS User ID: 37603035
aalg25@soton.ac.uk

1 Introduction

This report presents the implementation and evaluation of six neural network classifiers for the GTZAN music genre dataset [1], covering ten genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. Four image-based classifiers (Net1–Net4) operate on MEL spectrogram images, while two audio-based classifiers (Net5–Net6) operate on pre-extracted audio features. All models are implemented in PyTorch and trained under consistent experimental conditions to enable fair comparison.

2 Dataset and Preprocessing

The GTZAN dataset contains 1,000 audio clips (100 per genre), each 30 seconds long. Two modalities are used: PNG MEL spectrogram images (`images_original/`) for Net1–Net4, and the pre-extracted feature CSV file (`features_3_sec.csv`) for Net5–Net6.

2.1 Image Pipeline (Net1–Net4)

Spectrogram images are loaded using `torchvision.datasets.ImageFolder` and pre-processed with the following transforms: `resize` to 180×180 pixels using `torchvision.transforms.Resize`, conversion to tensor, and normalisation with mean and std of 0.5 across all three channels. The dataset is randomly split using `torch.utils.data.random_split` with a fixed seed (42) into training (70%, 700 samples), validation (20%, 200 samples), and test (10%, 100 samples) sets. A batch size of 32 is used across all image models.

2.2 Audio Pipeline (Net5–Net6)

The pre-extracted `features_3_sec.csv` file is used, which contains 57 audio features (MFCCs, chroma, spectral centroid, zero-crossing rate, etc.) computed over 3-second chunks, yielding 10 chunks per 30-second track and 10,000 rows in total. Features are standardised using `StandardScaler` and grouped

into sequences of 10 timesteps per song, producing 1,000 sequences of shape (10, 57). This design gives the LSTM a meaningful temporal structure to learn from. The same 70/20/10 split is applied with seed 42.

3 Neural Network Architectures

3.1 Net1 – Fully Connected Network

Net1 is a fully connected (FC) network with two hidden layers. `AdaptiveAvgPool2d(16, 16)` is applied before the FC layers, reducing the $180 \times 180 \times 3$ input from 97,200 to 768 features. Without this reduction, the weight matrix is intractably large and gradients vanish immediately. The architecture is: `AdaptiveAvgPool` → `Flatten` → `Linear(768, 512)` → `BatchNorm1d` → `ReLU` → `Dropout(0.4)` → `Linear(512, 256)` → `BatchNorm1d` → `ReLU` → `Dropout(0.4)` → `Linear(256, 10)`. Kaiming (He) initialisation is applied to all linear layers to improve gradient flow. Trained with Adam, $\text{lr}=10^{-4}$, weight decay= 10^{-4} .

3.2 Net2 – Convolutional Neural Network

Net2 follows the architecture in Figure 1 of the brief: two convolutional blocks each followed by `MaxPool`, then a fully connected head with 256 output features. `AdaptiveAvgPool2d(4, 4)` is applied before the classifier, reducing spatial dimensions from 45×45 to 4×4 and cutting FC parameters from 524,288 to 4,096. Trained with Adam, $\text{lr}=10^{-4}$, weight decay= 10^{-4} .

Block 1: `Conv(3→32)` → `ReLU` → `Conv(32→64)` → `ReLU` → `MaxPool(2×2)`

Block 2: `Conv(64→128)` → `ReLU` → `Conv(128→256)` → `ReLU` → `MaxPool(2×2)`

Classifier: `AdaptiveAvgPool(4×4)` → `Flatten` → `Linear(4096, 256)` → `ReLU` → `Dropout(0.5)` → `Linear(256, 10)`

3.3 Net3 – CNN with Batch Normalisation

Net3 has the same architecture as Net2 but with `BatchNorm2d` inserted after every convolutional layer,

before the activation. Batch normalisation normalises activations within each mini-batch, reducing internal covariate shift and stabilising training dynamics. All other hyperparameters remain identical to Net2.

3.4 Net4 – CNN with Batch Normalisation and RMSProp

Net4 is architecturally identical to Net3 but uses **RMSProp** instead of Adam. RMSProp adapts per-parameter learning rates by dividing by a running average of squared gradients, leading to different convergence behaviour on small datasets. Learning rate is 10^{-4} with weight decay 10^{-4} .

3.5 Net5 – Bidirectional LSTM

Net5 is a bidirectional LSTM classifier on sequences of pre-extracted audio features. Each song is represented as a sequence of 10 feature vectors of dimension 57. The architecture uses 2 LSTM layers with hidden size 128, bidirectional processing, and dropout of 0.3. Final hidden states from both directions are concatenated and passed to a classifier: *Linear*(256, 128) $\rightarrow$ *ReLU* $\rightarrow$ *Dropout*(0.3) $\rightarrow$ *Linear*(128, 10). Training uses Adam (lr= 10^{-3} , weight decay= 10^{-4}), gradient clipping (max norm=1.0), a ReduceLROnPlateau scheduler, and early stopping with patience of 20 epochs.

3.6 Net6 – Bidirectional LSTM with GAN Augmentation

Net6 uses the same LSTM architecture as Net5 but is trained on a GAN-augmented dataset. A Conditional GAN (cGAN) is trained for 100 epochs to generate synthetic feature sequences conditioned on genre label. The Generator maps Gaussian noise and a label embedding to a (10, 57) feature sequence via three fully connected layers with batch normalisation and Tanh output. The Discriminator classifies sequences as real or fake using a three-layer network with LeakyReLU activations and Sigmoid output. After GAN training, synthetic samples equal in number to the real training set are generated and combined with real data, doubling the effective training set size. Both Generator and Discriminator are trained with Adam (lr= 2×10^{-4} , betas=(0.5, 0.999)).

4 Efficiency Improvements

Several optimisations were adopted to improve both training efficiency and model performance. AdaptiveAvgPool2d was used in Net1–Net4 to reduce spatial feature map dimensions before fully connected layers, dramatically cutting parameter counts and training time without sacrificing representational capacity — reducing FC inputs in Net2 from 524,288 to 4,096 (a $128\times$ reduction). Kaiming (He) initialisation was

applied in Net1 to ensure stable gradient magnitudes from the first epoch, avoiding vanishing gradient issues present in the naive baseline. Gradient clipping (max norm = 1.0) was applied in Net5 and Net6 to prevent exploding gradients during LSTM backpropagation through time. A ReduceLROnPlateau scheduler was used in Net5 to adaptively reduce the learning rate when validation loss plateaued. Early stopping was adopted for Net5 and Net6 to prevent overfitting and eliminate unnecessary computation. For the audio pipeline, pre-extracted CSV features were used instead of raw waveforms, eliminating computationally expensive feature extraction at runtime and ensuring high-quality, reproducible input representations.

5 Results

All models are evaluated on the held-out test set. Table 1 summarises test accuracy across all six classifiers.

Table 1: Test accuracy (%) for all classifiers. Net1–Net4 report 50/100 epoch results; Net5–Net6 use early stopping.

Model	Architecture	Optimiser	Test Acc (%)
Net1	FC	Adam	50.50 / 59.41
Net2	CNN	Adam	51.49 / 58.42
Net3	CNN+BN	Adam	68.32 / 67.33
Net4	CNN+BN	RMSProp	67.33 / 70.30
Net5	LSTM	Adam	81.19
Net6	LSTM+GAN	Adam	81.19

6 Discussion

6.1 FC vs CNN

Net1 achieves 59.4% at 100 epochs, which is competitive given its lack of spatial feature extraction. However, significant overfitting is observed: training accuracy reaches 81.6% while validation accuracy plateaus at around 51%. This highlights a fundamental weakness of FC networks on image data — without spatial inductive bias, the model memorises training patterns rather than learning generalisable spectral features. Net2 (plain CNN) achieves a similar score (58.4%) at 100 epochs, performing below expectation due to unstable gradient magnitudes in the absence of batch normalisation.

6.2 Impact of Batch Normalisation

Batch normalisation in Net3 produces a substantial performance jump — 68.3% at 50 epochs versus 51.5% for Net2 at the same epoch count. This demonstrates BN’s well-known benefits: stabilised training dynamics, reduced internal covariate shift, and implicit regularisation. Net3 reaches peak accuracy at 50 epochs, suggesting BN accelerates convergence.

The marginal drop from 50 to 100 epochs (68.3% $\rightarrow$ 67.3%) indicates mild overfitting in later epochs when combined with Adam.

6.3 Adam vs RMSProp

Comparing Net3 and Net4 on identical architectures reveals that at 50 epochs both perform similarly (68.3% vs 67.3%). However, at 100 epochs Net4 with RMSProp continues to improve to 70.3% while Net3 with Adam stagnates. This suggests RMSProp maintains more stable gradient updates over longer training on this small dataset, avoiding the sharp over-adaptation that Adam's momentum terms can cause.

6.4 LSTM vs CNN

Net5 and Net6 both achieve 81.19%, substantially outperforming all CNN-based models. This is attributed to the high-quality pre-extracted features from `features_3_sec.csv`, which provide semantically rich temporal representations. The bidirectional LSTM effectively captures both forward and backward temporal dependencies across the 10 three-second chunks, a modality CNN models operating on 2D spectrograms cannot exploit directly.

6.5 Impact of GAN Augmentation

Net6 (LSTM + GAN) achieves the same test accuracy as Net5 (81.19%), suggesting that GAN-generated augmentation provides no measurable benefit at this scale. This is consistent with known limitations of GANs on small datasets — the generator may not produce sufficiently diverse samples to expand the training distribution meaningfully. Nevertheless, Net6 demonstrates a complete conditional GAN augmentation pipeline that could benefit from larger datasets or more training epochs.

6.6 Overall

The audio-based LSTM models (Net5, Net6) significantly outperform all image-based models, achieving 81.19% compared to the best CNN result of 70.30% (Net4). This indicates that structured temporal audio features capture genre-discriminative information more effectively than spatial patterns in MEL spectrograms at this dataset and model scale.

7 Reflection: Batch Normalisation in Deep Learning

7.1 Why the Topic is Important

Batch normalisation (BN), introduced by Ioffe and Szegedy [2], is one of the most impactful advances in deep learning. Before BN, training deep networks was extremely sensitive to

weight initialisation and learning rate choice. Small perturbations in early layers would cascade into large distribution shifts in later layers — a phenomenon known as *internal covariate shift*. BN addresses this by normalising layer inputs within each mini-batch to have zero mean and unit variance, then applying learnable scale (γ) and shift (β) parameters. This allows networks to train with higher learning rates, reduces dependence on careful initialisation, and acts as a mild regulariser. The practical importance of BN is directly observable in this work: Net3 outperforms Net2 by approximately 17 percentage points at 50 epochs despite being architecturally identical otherwise.

7.2 Current Deep Learning Technologies

Several BN variants have been developed to address its limitations. **Batch Normalisation** [2] normalises across the batch dimension and works well for CNNs with large batch sizes. **Layer Normalisation** normalises across features and is preferred for RNNs and Transformers where batch size may be 1. **Group Normalisation** divides channels into groups and normalises within each, remaining robust to small batch sizes. **Instance Normalisation** normalises per sample per channel and is commonly used in style transfer. **Spectral Normalisation** controls the Lipschitz constant of weight matrices and is widely used in GANs to stabilise discriminator training.

7.3 Implementation Capability

Batch Normalisation (Net3, Net4) and **Dropout** as regularisation have been successfully implemented in this coursework. **Layer Normalisation** is understood conceptually and could be implemented using `nn.LayerNorm` for the LSTM models. **Group Normalisation** and **Spectral Normalisation** present greater challenges; the primary difficulty lies in understanding their interaction with gradient dynamics and tuning associated hyperparameters such as number of groups and Lipschitz bound.

7.4 Positive and Negative Impacts, and Future Vision

The positive impacts of normalisation technologies are substantial: they have enabled training of extremely deep networks (ResNet, VGG, BERT) that would otherwise be unstable, and have democratised deep learning by reducing reliance on expert hyperparameter tuning. However, BN introduces non-trivial complications — it creates a dependency between samples in a mini-batch, breaking the i.i.d. assumption and causing models to behave differently at train and test time. This can harm performance when batch sizes are very small or in online/streaming settings. Looking forward, adaptive and learned normalisation — where normalisation parameters are dynamically conditioned on input content rather than fixed per-layer — will become increasingly important in multimodal and generative models.

Technologies such as Adaptive Instance Normalisation (AdaIN) already hint at this direction, and the integration of normalisation strategies with neural architecture search may eventually automate what is currently a manual design decision.

References

- [1] G. Tzanetakis and P. Cook, *Musical Genre Classification of Audio Signals*, IEEE Transactions on Speech and Audio Processing, 2002. 1
- [2] S. Ioffe and C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, ICML, 2015. 3